

Memory Management

Module 5

Operating Systems

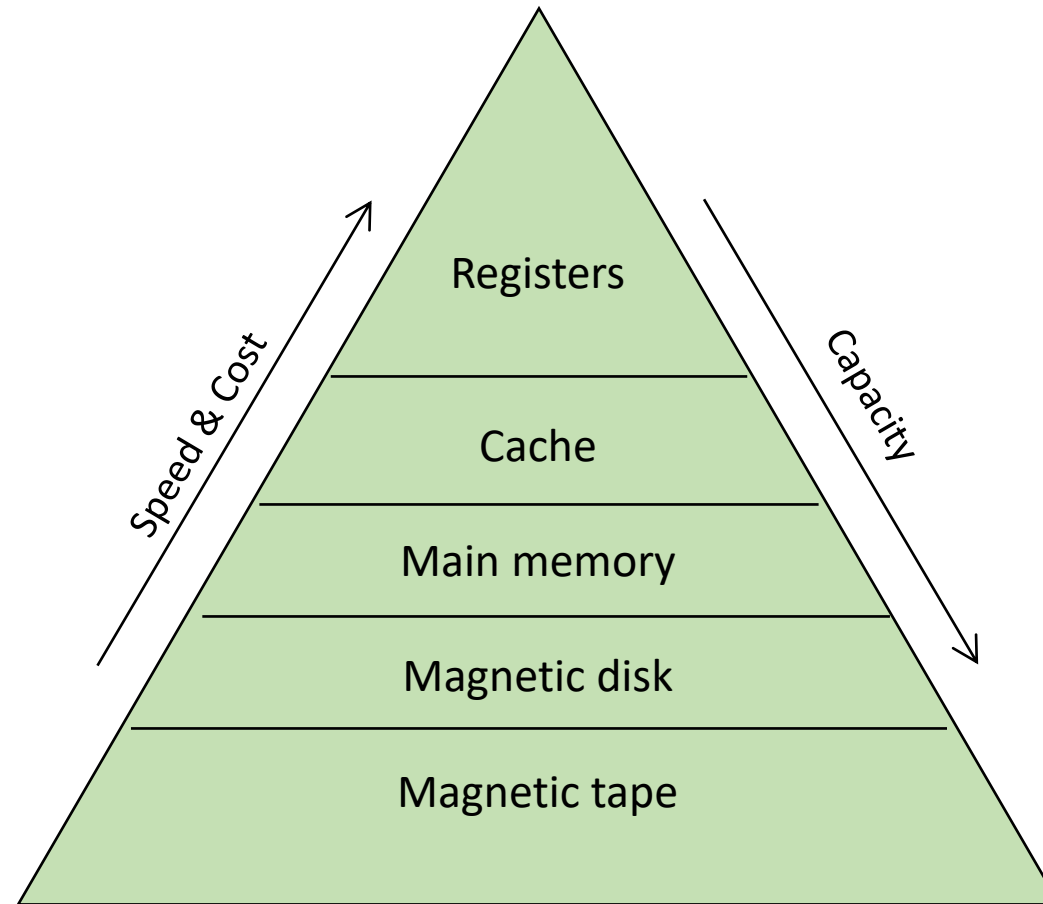
Chapter Outline

- Memory
 - Memory hierarchy
 - Process loading and swapping
 - Memory management
 - Memory allocation methods
- Storage management strategies
- Contiguous memory allocation
- Non-contiguous memory allocation
 - Paging
 - Simple paging
 - Implementation of paging
 - Segmentation
 - Simple segmentation
 - Segment addressing

Memory

- Used for storage and retrieval of data and instructions
- Essential component
- Logically memory system can be divided into three groups
 - Internal memory
 - Set of registers in processor
 - they hold temporary results during execution
 - Main memory
 - Communicates directly with CPU
 - Secondary (external memory)
 - Peripherals storage devices such as disk and tape etc

Memory Hierarchy



Memory Hierarchy

- Registers
 - are fastest, smallest and most expensive
- Main memory
 - It is principal memory
 - It is generally extended with a higher speed smaller cache
 - Used to improve movement of data between main memory and processor registers
 - Usually not visible to user or programmer or indeed to the processor
- External devices are permanent storage
 - Also referred as secondary or auxiliary memory
 - e.g. magnetic disk, tape, hard disk

Process Loading & Swapping

- Process loading
 - Transfer of a process from secondary storage to main memory
- Process Swapping
 - Transfer of a process from memory to secondary storage

Memory Management

- Memory is a volatile array of words or bytes, shared by the CPU and I/O devices
- Programs must be brought into memory and place in form of a process to be executed
- The operating system manages the memory activities
 - Keep track of which parts of memory are currently being used and by whom
 - Decides which process(s) to load when memory space becomes available
 - Allocate and de-allocate memory space to processes as required

Address Binding

- Address binding is the process of mapping logical addresses generated by a program to physical addresses in main memory, carried out at compile-time, load-time, or execution-time.

CPU (logical address)



MMU (binding/mapping)



RAM (physical address)

Address Binding

1. Compile-Time Address Binding

- Binding happens **during program compilation**.
- Logical address = physical address.
- Program must load at a **specific memory location**.
- If location changes, program must be **recompiled**.
- Rarely used.

Example: Embedded systems.

2. Load-Time Address Binding

- Binding happens **when the program is loaded into memory**.
- Compiler generates **relocatable code**.
- Physical address decided by OS loader.
- If address changes → **no need to recompile**, just reload.

Address Binding

3. Execution-Time Address Binding (Run-Time Binding)

- Binding happens **while program is running**.
- Requires **hardware support** such as **MMU (Memory Management Unit)**.
- Allows processes to move in memory during execution (swapping/relocation).
- Used in **modern OS** and **virtual memory** systems.

Important Concepts

- **Logical Address:** Generated by CPU (virtual address).
- **Physical Address:** Actual location in RAM.
- **MMU (Memory Management Unit):** Hardware that converts logical → physical address at runtime.

Logical vs. Physical Address Space

- Logical address
 - Generated by CPU
 - Also referred to as virtual address
- Physical address
 - Address seen by the memory unit
- Compile and load time binding
 - Logical = physical memory
- Execution time binding
 - Logical is virtual memory, i.e. does not necessarily coincide with physical memory

Virtual Addresses

- There exist a set of memory addresses that programs can produce
- These program generated addresses are called virtual or logical addresses and form virtual address space
- Physical addresses are the addresses seen by the memory unit
- **Logical and physical addresses are same at compile time**
- **Logical and physical addresses differ in execution time**

Why Virtual Address is Used?

- To give each process its **own memory space**
- To enable **memory protection**
- To support **paging & segmentation**
- To allow **programs larger than RAM** (virtual memory)
- To allow **relocation & swapping**

Memory Management Unit

- Hardware device that maps virtual address to physical address
- Use a base or relocation register
 - The value in relocation register is added to every address generated by a user process at the time it is sent to memory
- The user programs deals with logical addresses
 - It never sees the real physical addresses

Why is MMU Needed?

MMU provides:

- **Virtual memory**
- **Memory protection**
- **Relocation**
- **Paging & segmentation**
- **Swapping support**
- **Isolation between processes**

Without MMU, CPU would directly use physical addresses → no security, no virtual memory.

Memory Management Unit

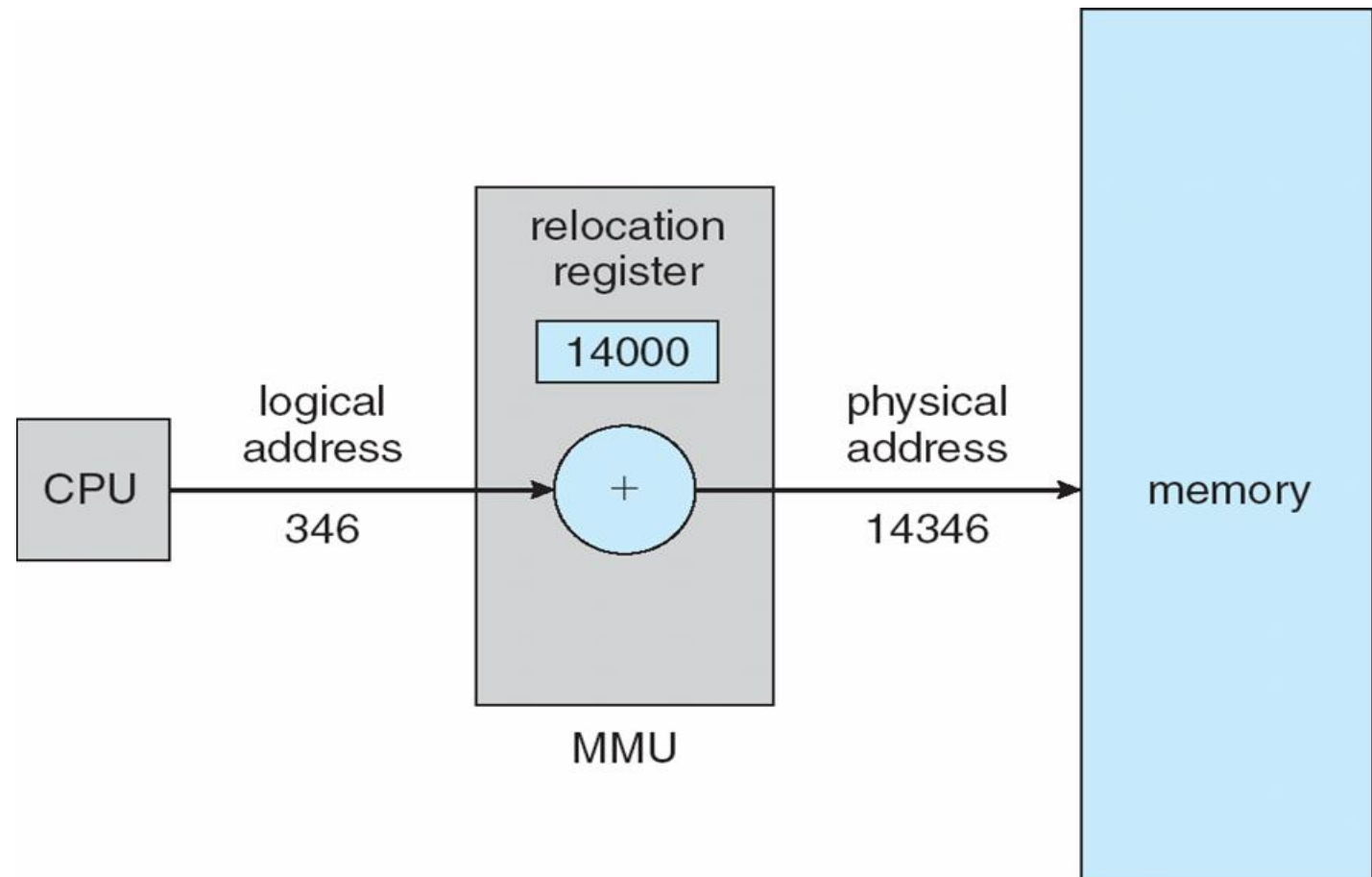
- Program executes an instruction that references address 346 — this is the **logical address** (an offset within the program's address space).

- MMU reads the **relocation/base register** value = 14000. That value is the physical address where the process's logical address 0 is mapped.

- MMU computes:

Physical address = Base + Logical $\rightarrow 14000 + 346 = 14346$.

- The memory system uses physical address 14346 to fetch/store the requested data.



Memory Management Requirements

1. Relocation

- Main memory is shared between processes
- Swap active processes in and out to provide large pool of ready processes
 - Quite difficult to declare when swapped out process will be placed back
- A process image occupies a contiguous region of main memory
- To run this process, the processor hardware and operating system must be able to translate the memory reference found in code, into physical memory address

Memory Management Requirements

2. Protection

- Each process must be protected by unwanted interference by other processes
- Without prior permission, a process should not refer to other process
- All memory references generated by a process must be checked at run time
 - to ensure that they refer only to the allocated memory space
 - The processor must abort such instructions when executed
- Thus, the memory protection requirement must be satisfied by the processor (hardware) rather than operating system
 - Since OS cannot anticipate all the memory references a program will make
 - Even if such participations are possible, it would be prohibitively time consuming to screen each program in advance for possible memory reference violation

Memory Management Requirements

3. Sharing

- Allow several processes to access the same area of main memory
- Processes that are cooperating on some tasks may need to share access the same data structure
- Allow controlled access to share areas of main memory without compromising essential protection

Memory Management Requirements

4. Local organization

- Main memory is organized as a linear or one-dimensional address space that consist of a sequence of bytes or words
- Secondary memory at its physical level is similarly organized
- It does not correspond to the way programs are constructed
- Mostly programs are organized into modules
 - Some of which are un-modifiable (read or executable only)
 - Some of which contain that can be modified

Memory Management Requirements

5. Physical organization

- Secondary memory of large capacity provides long term storage of program and data
- While a smaller main memory holds programs and data currently in use
- The organization of flow of information between main memory and secondary memory is a major system concern

Program Loading into Memory

- The core task of any memory management system is to bring and manage programs in main memory
- In almost all modern multiprogramming systems, this task involves sophisticated scheme known as virtual memory
- Virtual memory is in turn based on the use of one or both of two basic techniques
 - Segmentation
 - Paging

Memory Management Techniques

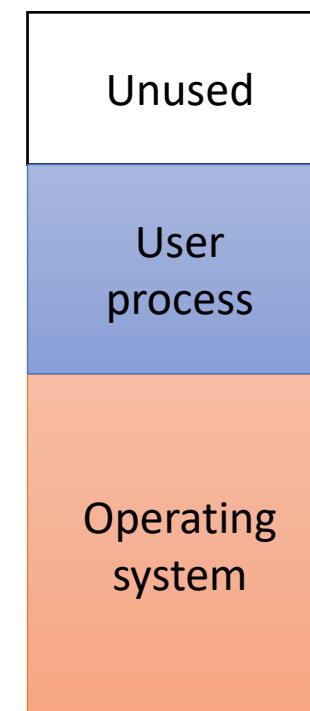
Different schemes used in operating systems to allocate memory to processes

- Contiguous Memory Allocation
 - Single process system
 - Fixed size (static) partition memory
 - Variable sized (dynamic) partition memory
- Non-contiguous Memory Allocation
 - Paging
 - Segmentation
 - Variations of above
 - Paged segmentation
 - Demand paging
 - Demand segmentation
 - Virtual memory

Contiguous Memory Allocation

Single Process System

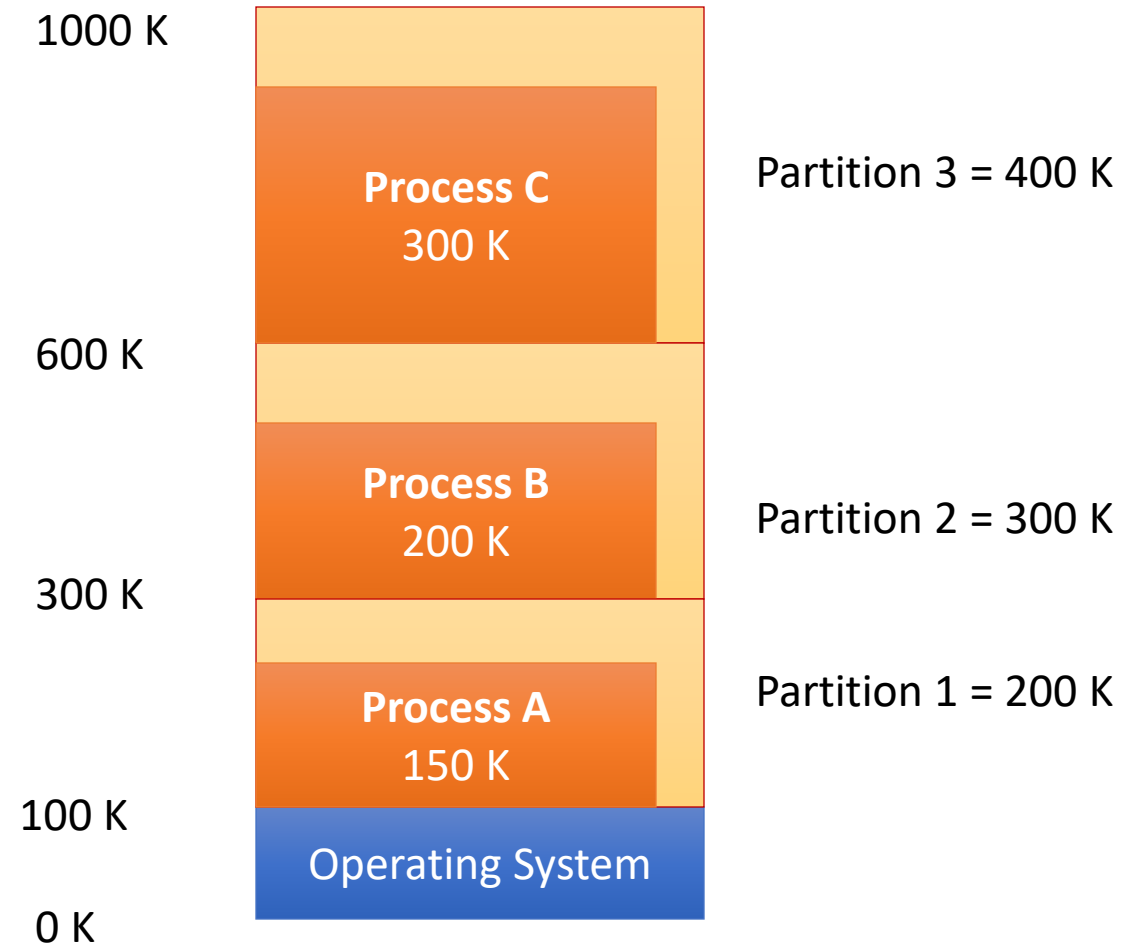
- Memory is divided into two partitions
 - One for user process and
 - One for the operating system
- Most common to place the operating system in low memory
- Run one process at a time in high memory
- Memory management is very simple
- Example;
 - MS-DOS system is an example of this system



Fixed Sized Partition Memory

- Memory is divided into several fixed partitions
- Each partition may contain exactly one process
- When partition is free, another process is loaded into free partition
- Waste considerable amount of main memory when processes are smaller than allocated partition
 - Called internal fragmentation
- First employed in multiprogramming computers
- Used in IBM OS/360
- Disadvantages
 - Prevent a process being run due to unavailability of sufficient size partition
 - Internal fragmentation wastes space

Fixed Sized Partition Memory



Variable-Sized Partition Memory

- The processes would be given as much storage as they required
- Processes are loaded into consecutive areas until the memory is filled
- When a partition is free, another process is loaded into free partition
- External fragmentation
 - It refers to the distribution of the free memory space, out-with any process allocation
- What are the problems here?
 - What if the size of the vacant partition(s) is smaller than the size of the incoming process?
 - A new process cannot be started
- Solution
 - Coalescing
 - It refers to combine two or more adjacent holes to accommodate a large incoming process
 - Difficulty: Coalescing is a time-consuming procedure

Variable-Sized Partition Memory

- Variable partition scheme has been used successfully in many computers
- Overheads
 - In variable partition, the system must keep track of
 - the position
 - size of each hole
 - taking account of the effect of coalescing
 - The solution is to use linked list, which contains
 - Pointer to the start of each hole and
 - The hole size

Variable-Sized Partition Memory

- Variable-sized partition with compaction
 - Compaction is a process of physically moving memory resident processes to close-up the holes
 - Difficulty;
 - It is a time-consuming procedure
 - All processes are suspended while reshuffling takes place
 - Not feasible in time critical system
 - Compaction may be performed
 - As soon as any process terminates
 - When new process cannot load due to fragmentation
 - At fixed intervals
 - When the user decide to do

Storage Management Strategies

- Storage management strategies are used to obtain best possible use of main memory
- The strategies are divided into following categories
 - Fetch strategies
 - Placement strategies
 - Replacement strategies

Storage Management Strategies

Fetch Strategies

- When to obtain the next process for transfer to main memory from secondary memory
- It could be done by
 - Demand fetch strategies
 - In this next process is brought to main memory when it is referenced by running program
 - Anticipatory fetch strategies
 - In this system predicts program's needs and attempt to load the appropriate process

Storage Management Strategies

Placement Strategies

- Placement strategies concerned with determining where in main memory to place an incoming process
- Three placement strategies are;
 - Best fit
 - Places the process in smallest of the available holes
 - Intuitively appealing
 - First fit
 - Places the process in first available hole, which is large enough to hold it
 - Low overheads
 - Worst fit
 - Places the process in the largest available hole
 - Another process can be accommodated if hole is big enough
- Example;
 - A variable partition memory with hole sizes in order 20k, 15k, 40k, 60k, 10k, 25k
 - What if a new process of 25k is to be loaded

Storage Management Strategies

Replacement Strategies

- They are concerned with determining which process to displace to make room for incoming process
- Decision is made on different factors

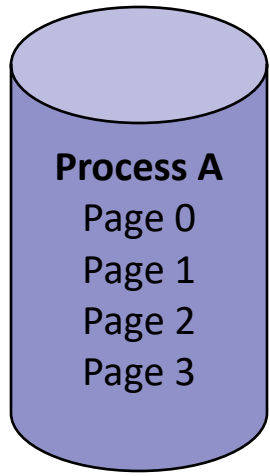
Non-contiguous Memory Allocation

Paging & Segmentation

Paging

- Fixed sized and variable sized partitions are insufficient due to
 - Internal fragmentation
 - External fragmentation
- Paging removes these problems
 - Main memory is partitioned into equal fixed-size chunks called **frames**
 - Relatively small in size
 - Each process is also divided into fixed size chunks called **pages**
 - Same size as memory frames
 - Page size is typically power of 2 (typically 4K-8K)
 - depending on the computer architecture
 - No need of contiguous frames for a process or even in correct order

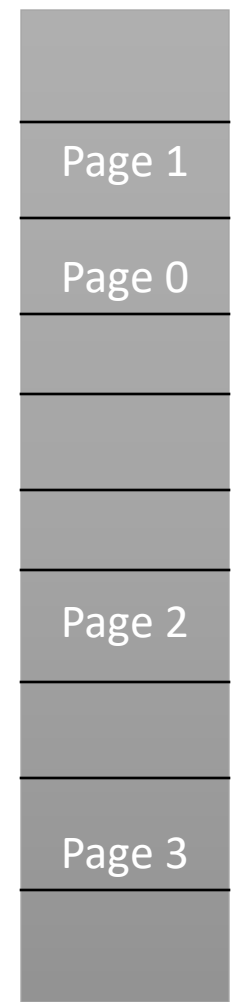
Paging



Free frame list

- 14
- 13
- 18
- 20
- 15

- 12
- 13
- 14
- 15
- 16
- 17
- 18
- 19
- 20
- 21



Process A

Page Table

0	14
1	13
2	18
3	20

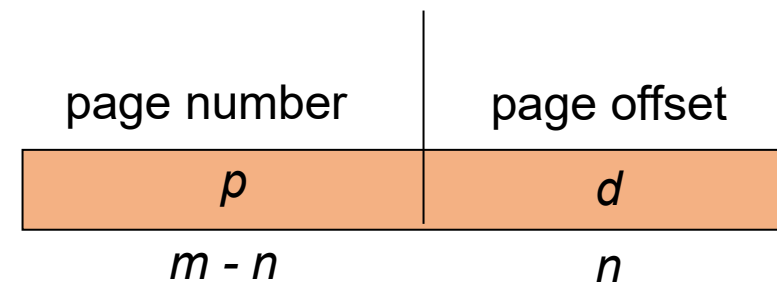
- 12
- 13
- 14
- 15
- 16
- 17
- 18
- 19
- 20
- 21



Page Addressing

1. What is Page Addressing?

- In **paging**, memory addresses are divided into:
- **Page Number (p)**: Identifies which **page** of the process the address belongs to.
- **Page Offset (d)**: Identifies the **exact location within that page**.
- Logical addresses are translated to **physical addresses** using the **page table**.



2. Logical Address Structure

Suppose a process has:

- Logical address space: 0 to 4095 (i.e., 12-bit address)
- Page size: 1024 bytes (2^{10})

Then:

- **Page number (p)**: 2 bits (since 4 pages needed: 0–3)
- **Page offset (d)**: 10 bits (0–1023 within each page)

Example:

Logical address = 2500

Page size = 1024

Page number = $2500 / 1024 = 2$

Page offset = $2500 \% 1024 = 452$

Address Translation Using Page Table

Steps:

1. CPU generates **logical address** (p, d).
2. **Page table** is consulted to get the **frame number** (f) for page p.
3. **Physical address** = (frame number × frame size) + page offset

Example:

•Page table:

Page	Frame
0	5
1	2
2	8
3	9

- Logical address: Page 2, Offset 452
- Frame number = 8
- Frame size = 1024 → Physical address = $8 \times 1024 + 452 = 8704$

Information:

- Logical Address:** (Page Number, Offset)
- Physical Address:** (Frame Number, Offset)
- Translation:** Physical Address = Frame × Frame Size + Offset

Paging allows **non-contiguous allocation** while **eliminating external fragmentation**, and page addressing makes it **easy to locate data** in memory.

Page Address Translation

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

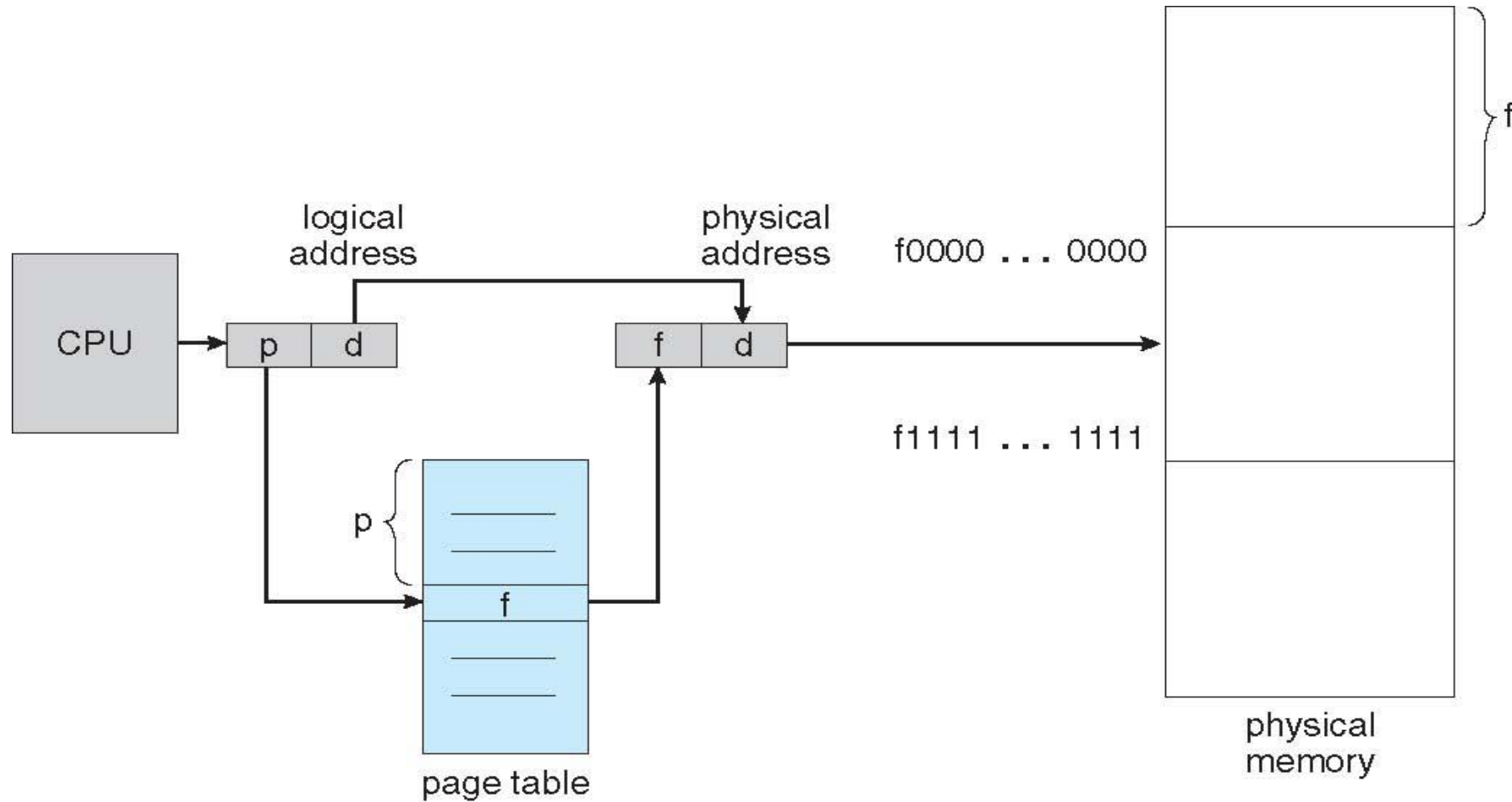
0	5
1	6
2	1
3	2

page table

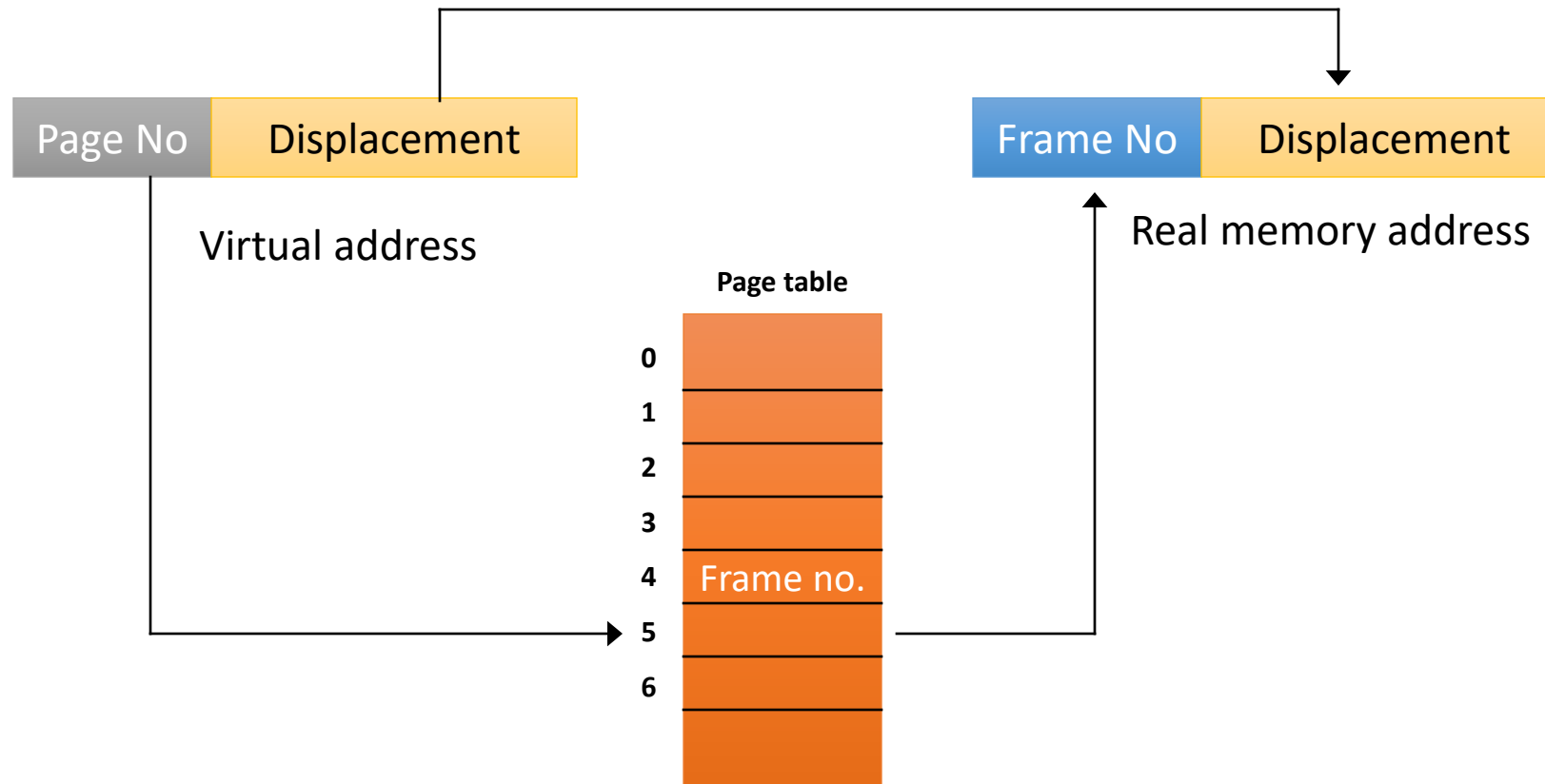
0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g h
28	

physical memory

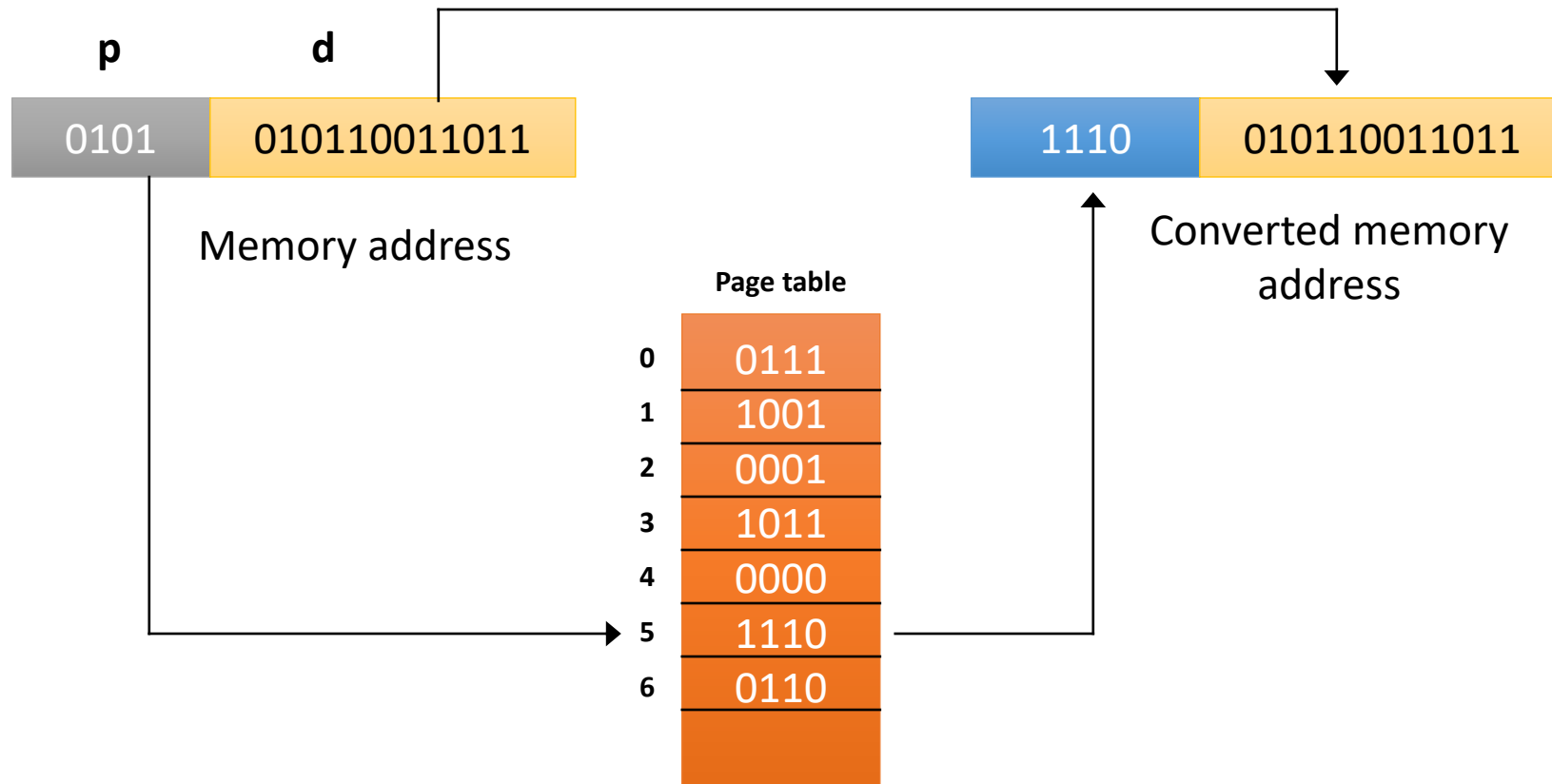
Page Address Translation



Page Address Translation



Page Address Translation



Internal Fragmentation

Internal Fragmentation occurs when:

Allocated memory is **slightly larger** than required memory, and the **unused space lies inside the allocated block**.

In **paging**, this happens because:

- Pages have **fixed size**
- Process size is **not always a multiple of page size**
- The **last page** is usually **partially filled**

Important:

Only the **last page** of a process can cause internal fragmentation.

Why Internal Fragmentation Happens in Paging?

Paging divides memory into **equal-sized frames**, so:

- Even if a process needs **1 extra byte**, the OS must allocate **one full page**

Remaining unused bytes in that page = **internal fragmentation**

Example: Internal Fragmentation

Given:

- Page size = 2048 bytes
- Process size = 72,766 bytes

Solution:

- **Step 1: Calculate Number of Pages Needed**

Divide process size by page size:

$$72,766 \div 2,048 = 35.53$$

Since partial pages are **not allowed**, round **up**:

Total pages = 36 pages

This means:

- 35 pages are **fully used**
- 1 page is **partially used**

Step 2: Calculate Used Bytes in Last Page

Bytes used in last page:

$$\text{Used bytes} = 72,766 - (35 \times 2,048)$$

$$\text{Used bytes} = 72,766 - 71,680$$

$$\text{Used bytes} = 1,086 \text{ bytes}$$

General Formula:

$$\text{Internal Fragmentation} = \text{Page Size} - (\text{Process Size mod Page Size})$$

Step 3: Calculate Internal Fragmentation

Internal Fragmentation:

$$\begin{aligned} \text{Internal Fragmentation} &= \text{Page size} - \text{Used bytes} \\ &= 2,048 - 1,086 \\ &= 962 \text{ bytes} \end{aligned}$$

$$\text{Internal Fragmentation} = 962 \text{ bytes}$$

Worst Case & Average Case Fragmentation

Worst Case Fragmentation:

- 1 frame – 1 byte

Example:

- Page size = 2048 bytes
- Process uses only 1 byte in last page
- Waste = 2047 bytes

Average Fragmentation:

- $\frac{1}{2}$ frame size

Example:

- Page size = 2048 bytes
- Average waste = 1024 bytes

Practice Question:

Page size = 1 KB, process size = 10.5 KB
Find internal fragmentation.

Approach:

1. Convert to bytes
2. Calculate pages
3. Subtract used space
4. Apply formula

Solution: Internal Fragmentation in Paging

- **Given:**
- **Page size = 1 KB**
- **Process size = 10.5 KB**

Step 1: Convert to Bytes

1 KB = 1024 bytes

Process Size:

$= 10.5 \times 1024 = 10,752$ bytes

Step 2: Calculate Number of Pages Required

Number of pages = Process size / Page size

$= 10,752 / 1,024$

$= 10.5$ pages

Since a process cannot occupy a fraction of a page:

Total pages allocated = 11 pages

Step 3: Calculate Used Space in the Last Page

Space used by 10 full pages:

$= 10 \times 1,024 = 10,240$ bytes

Remaining data in last page:

$= 10,752 - 10,240$

$= 512$ bytes

Step 4: Calculate Internal Fragmentation

Internal Fragmentation = Page size – Used space in last page

$= 1,024 - 512$

$= 512$ bytes

Internal Fragmentation = 512 bytes

Direct Solution:

Internal Fragmentation = Page size – (Process size mod Page size)

$= 1024 - (10,752 \bmod 1,024)$

$= 1024 - 512$

$= 512$ bytes

Page Table Structure

- Hierarchical Page Tables
- Hashed Page Tables
- Inverted Page Tables



Hierarchical Page Tables

- **Multilevel (Hierarchical) Page Table**
- Instead of one large page table, the table is **divided into multiple levels**. Each level points to the next level.

Why it is needed

- Reduces memory usage
- Allocates page tables **only when required**
- Suitable for **sparse address spaces**

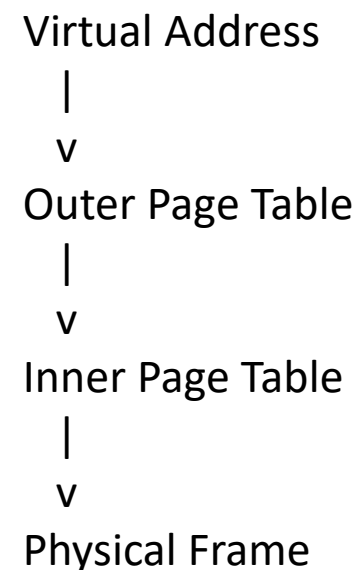
How it works (Conceptually)

- Virtual address is split into:
 - Outer page number
 - Inner page number
 - Offset
- Outer table entry points to an inner table
- Inner table entry gives the frame number

Key Characteristics

- Saves memory compared to single-level paging
- Increases **address translation time**
- Commonly used in **32-bit and 64-bit systems**

Diagram



Hashed Page Tables

The virtual page number is passed through a **hash function** to locate the page table entry.

Why it is needed

- Efficient for **very large address spaces**
- Reduces size of page table
- Works well when virtual addresses are **sparse**

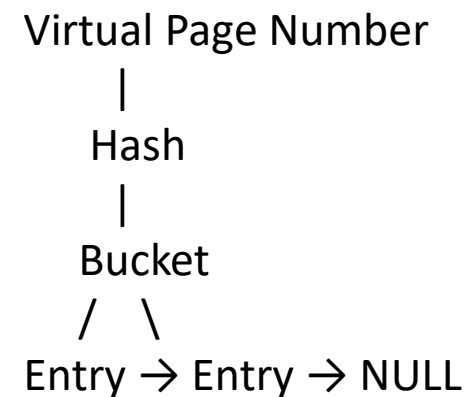
How it works

- Hash(Virtual Page Number) → index
- At that index, a **linked list** of entries may exist
- Each entry stores:
 - Virtual page number
 - Physical frame number
 - Pointer to next entry

Key Characteristics

- Faster lookup for large address spaces
- Hash collisions may occur
- Mainly used in **64-bit systems**

Diagram:



Inverted Page Table

Instead of one page table **per process**, there is **one global page table** for the entire system.

Why it is needed

- Drastically reduces memory overhead
- Page table size depends on **physical memory**, not virtual memory

How it works

- One entry per **physical frame**
- Each entry stores:
 - Process ID
 - Virtual page number
- To find a page, the system searches the table (usually using hashing)

Key Characteristics

- Very memory-efficient
- Sharing memory becomes complex

Diagram:

Physical Frame Table

+-----+			
PID	VPN	Frame	
+-----+			
P1	3	0	
P2	5	1	
P1	8	2	

Segmentation

Segmentation is a **non-contiguous memory allocation technique** in which:

- Segmentation divides a process into variable length chunks called segments
- Almost similar to variable sized partitions
- Segment's length is determined by the system
- The segments defined by a programmer reflect the modular structure of the process
 - e.g. data in one segment and code in another segments

Program

- └— Code Segment
- └— Data Segment
- └— Stack Segment
- └— Heap Segment

Each segment:

Has its **own size**

Can be placed **anywhere in memory**

Can grow or shrink independently

Segment Addressing

- The segment address consist of two parts
 - The segment number s and
 - The displacement d within that segment
- A process segment table entries specify the base address and the segment size
- The address conversion process is similar to the paging system
- A segmented address reference requires the following steps
 - Extract the segment number and the displacement from the logical address
 - Use the segment number to obtain the segment base address and length from segment table

Segment Table

+-----+

| Seg | Base | Limit | Prot |

+-----+

| 0 | 1400 | 1000 | R-X | ← Code

| 1 | 2400 | 800 | RW- | ← Data

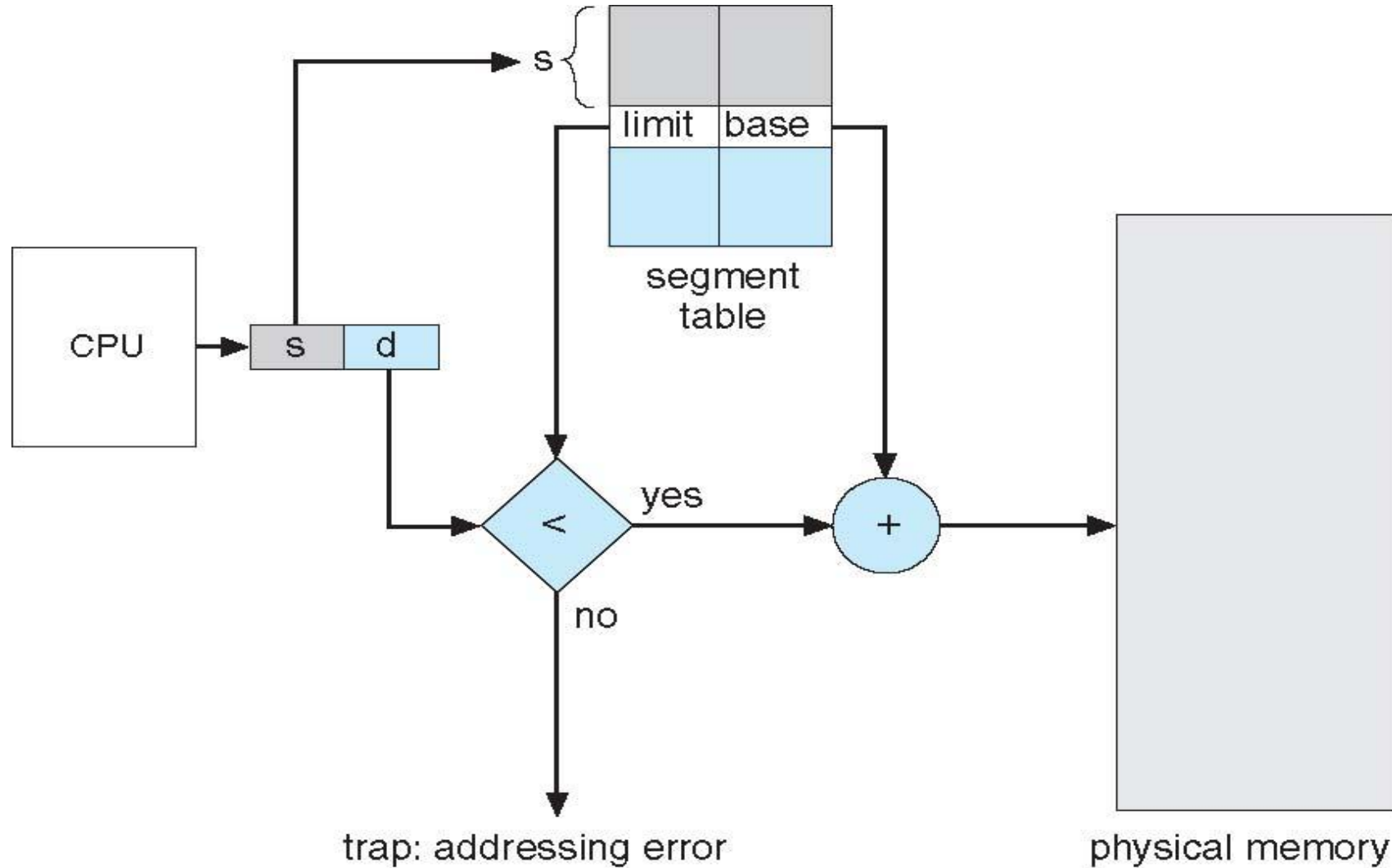
| 2 | 3200 | 600 | RW- | ← Stack

| 3 | 4300 | 1200 | RW- | ← Heap

- Check that the offset is not greater than the given length; if so, an invalid address is signaled **[$d < \text{Limit} ?$] → Invalid**
- Generate the required physical address by adding the offset to the base address

Physical Address = Base + Offset

Segment Addressing



Example

Program Example

Suppose a program is divided into **4 logical parts**:

Segment Table (Given by OS)

Segment No	Segment Name	Size
0	Code	1000 bytes
1	Data	600 bytes
2	Stack	400 bytes
3	Heap	800 bytes

Segment No	Base Address	Limit (Size)
0	1400	1000
1	2400	600
2	3200	400
3	4300	800

Suppose CPU generates this logical address:

(Segment Number = 1 , Offset = 200)

Segment 1 → **Data segment**

Offset 200 → 200 bytes inside data segment

Address Translation Step-by-Step

Step 1: Find Segment Table Entry

- Segment 1 →
Base = 2400
Limit = 600

Step 2: Check Offset

- Offset < Limit ?
- 200 < 600 (Valid)

Step 3: Calculate Physical Address

Physical Address = Base + Offset
= 2400 + 200
= 2600
Physical Address = 2600

Invalid Address Example

Logical Address:

(Segment = 1 , Offset = 700)

Check:

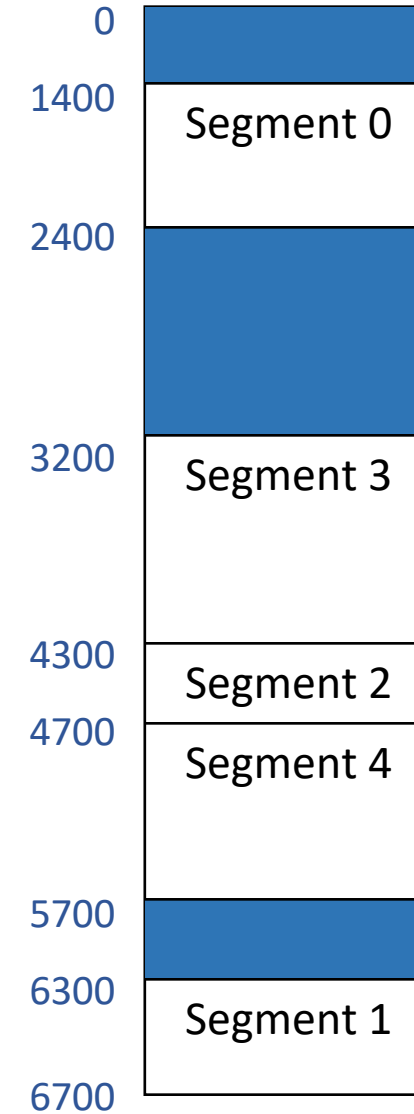
700 < 600 (Invalid)

This causes:

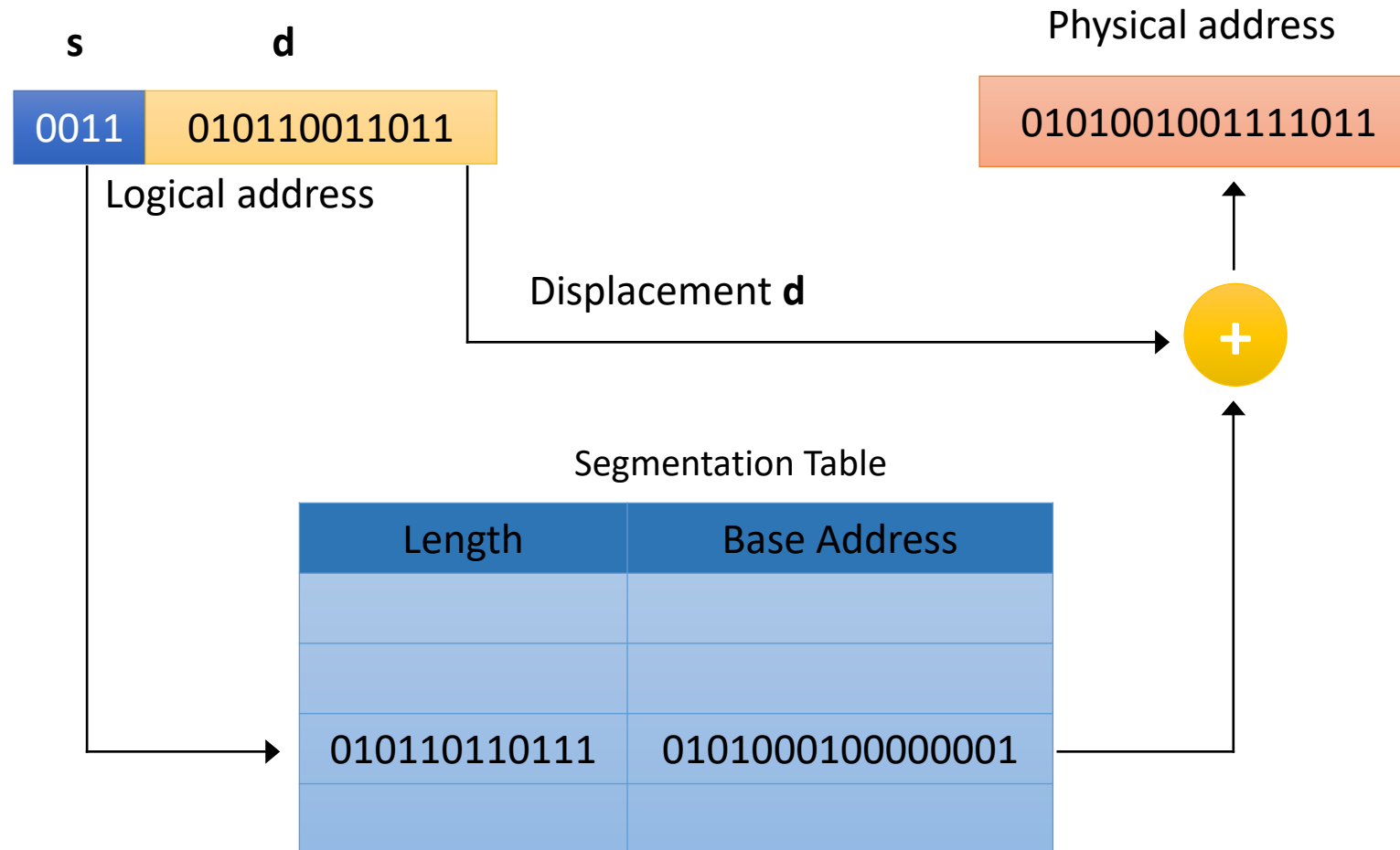
Segmentation Fault

Segment Addressing

Segment Table		
	Limit	Base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700



Segment Address Translation



Practice Example

- **Given Program Segments**
- A process is divided into **5 segments** as follows:

Segment Table (Given)

Segment No	Segment Name	Size (bytes)	Segment No	Base Address	Limit
0	Code	1200	0	1000	1200
1	Data	800	1	2500	800
2	Stack	500	2	3500	500
3	Heap	1000	3	4500	1000
4	Library	700	4	6000	700

- Find the **physical address** for the logical address:
(Segment = 3 , Offset = 600)
- Check whether the following logical address is **valid or invalid**:
(Segment = 2 , Offset = 520)

Paging vs. Segmentation

- Paging use small chunks 'pages' of same size
- Segmentation use chunks 'segments' of variable sizes
- Paging avoids fragmentation
- Segmentation also improves allocation while preserving the process structure
- Paging system is independent of programmer while segmentation reflects the logical structure of a process

Paging and Segmentation

- A process can be loaded in separate parts (using segmentation or paging)
- Memory address references can be translated dynamically at run time
- Not necessary to load all pages of a process into memory
 - Load portions being referenced at time of execution
 - Rest is retained on secondary storage